Satellite Land-Use Classification & Change Detection

Project Report

1. Why I Built This

Satellite imagery is one of those areas where a small model can genuinely save someone hours of manual work. Governments, insurers, and researchers already use satellite tiles to track deforestation, urban sprawl, and crop health, but a lot of that review is still done by eye. The goal of this project was to build a small but complete pipeline that could (1) look at a satellite tile and say what kind of land it is showing, and (2) compare two tiles of the same location taken at different times and flag whether something meaningfully changed.

I chose a transfer-learning approach rather than training a network from scratch, because with a dataset the size of EuroSAT, a pretrained backbone gets you to a usable accuracy far faster, and it's a more realistic reflection of how this kind of project would actually be built in practice.

2. The Data

The core dataset is EuroSAT, a well-known benchmark of about 27,000 Sentinel-2 satellite tiles, split across 10 land-use classes: AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River, and SeaLake. Alongside it, the project also pulls in the UC Merced Land Use dataset (2,100 images, 21 classes) as a completely separate holdout set, so I could sanity-check whether the model was learning something that generalizes beyond EuroSAT's specific imagery style, rather than just memorizing it.

One detail I paid particular attention to was how the data gets split into train/validation/test sets. EuroSAT doesn't ship with real geographic coordinates in the version available through torchvision, so a naive random split can accidentally put two tiles from the same physical location in different splits — which quietly leaks information and makes the model look better than it really is. To avoid that, the project hashes each image into one of a 10x10 grid of synthetic geographic blocks and splits by block, not by individual image, so no two tiles from the same neighborhood end up split across train and validation.

3. How I Approached the Modeling

I built two models to compare against each other:

- A baseline: a simple 3-layer convolutional network trained from scratch, mainly to have an honest lower bound to compare against.
- A fine-tuned model: a ResNet-18 backbone pretrained on ImageNet, adapted to EuroSAT using a two-phase fine-tuning strategy.

The two-phase part is the piece I'm most pleased with. In phase 1, the ResNet backbone is completely frozen and only a new classification head is trained on top of it — this lets the head adjust to the new classes without disturbing the pretrained features. In phase 2, the last two residual blocks (layer3 and

layer4) are unfrozen, the learning rate is dropped by 10x, and training continues for a few more epochs. The idea is to let the network's higher-level features adapt to satellite imagery specifically — which looks quite different from the everyday photos ResNet was originally trained on — without wrecking the low-level filters that already work well.

4. What Actually Happened When I Trained It

Here's the real training log from the run I did on my machine (backbone: ResNet-18, phase 1: 3 epochs, phase 2: 5 epochs, run on CPU):

Phase	Epoch	Train Loss	Val Accuracy	Notes
1 (frozen backbone)	1/3	0.8830	76.6%	head only
1	2/3	0.6641	80.0%	
1	3/3	0.6410	79.9%	plateauing
2 (unfrozen layer3/4)	1/5	0.3892	92.5%	big jump
2	2/5	0.2392	94.0%	
2	3/5	0.1972	95.2%	best epoch
2	4/5	0.1755	95.1%	slight dip
2	5/5	0.1500	95.1%	checkpoint saved

A few things stood out to me in these numbers. First, phase 1 alone (frozen backbone, head-only training) got to about 80% validation accuracy, which is a solid starting point, but you can see it plateauing across epochs 2 and 3 — the head can only do so much on its own. Once phase 2 unfroze the deeper layers, validation accuracy jumped to over 92% in a single epoch and climbed to a peak of about 95.2% by the third epoch of phase 2. That kind of jump is a good sign — it means the frozen backbone genuinely wasn't a great fit for satellite imagery out of the box, and letting it adapt made a real difference.

The other thing I noticed is that accuracy very slightly dipped and flattened out after that peak (95.2% -> 95.1% -> 95.1%), even as training loss kept dropping. That's a classic early sign of the model starting to overfit — it's continuing to fit the training set more precisely, but validation performance has essentially plateaued. It's not a serious problem at this scale, but if I were pushing this further, I'd either add early stopping around that third phase-2 epoch or bring in some data augmentation to squeeze out a bit more headroom.

5. Building the Change-Detection Dashboard

The second half of the project is a Streamlit dashboard that takes the trained model and turns it into something you can actually interact with, rather than just a checkpoint file. You upload a 'before' tile and an 'after' tile, and the app does three things:

- Classifies each tile independently and shows the predicted class and confidence for both (Prediction T1 / Prediction T2).
- Extracts a 512-dimensional embedding for each image from the fine-tuned network and compares them using cosine similarity — this is the actual 'change detection' signal. Two tiles of the same unchanged location should produce very similar embeddings; a location that's been built over, cleared, or flooded should not.

- Lets you choose how strict the change flag should be, via three preset operating points — High Recall, Balanced, and High Precision — since in a real deployment, a conservation team monitoring for illegal logging cares about different trade-offs than an insurer double-checking a claim.

The part I spent the most time refining was the heatmap. My first version just painted the entire image a single flat color based on one overall similarity score, which wasn't very informative — it told you *whether* something changed, but not *where*. I rebuilt it to slice each tile into an 8x8 grid of patches, embed each patch individually, and compute similarity patch-by-patch, so the heatmap now actually varies across the image and highlights the specific regions that changed the most. You can view it either blended on top of the actual photo (with an adjustable overlay strength) or as a standalone color map.

I also went back and reworked the visual design of the dashboard a couple of times — cleaning up deprecated Streamlit parameters, restructuring the layout into a clearer top-to-bottom flow (upload, then previews, then predictions, then the similarity/threshold/status readout, then the heatmap), and adjusting the color palette to a lighter, more professional look with navy text and blue accents rather than the default dark theme.

6. What's Still Pending

In the interest of being upfront: the fine-tuned checkpoint is trained and the dashboard is fully working end-to-end, but I haven't yet run every script in the pipeline. A few pieces are built and ready to run, but don't have results filled in yet:

- Formal evaluation (evaluation/evaluate.py) — proper confusion matrices and per-class F1 scores on both the EuroSAT validation set and the UC Merced holdout, rather than just the validation accuracy printed during training.
- The spatial leakage experiment (evaluation/spatial_leakage.py) — this trains the same model under both a random split and the block-based split described in Section 2, to put an actual number on how much a naive split would have inflated the accuracy.
- Error analysis (evaluation/error_analysis.py) — pulling out the model's most confidently wrong predictions to understand what it's actually struggling with (my guess, based on the classes involved, is that Highway/River and AnnualCrop/PermanentCrop are the most likely sources of confusion, since they can look visually similar from directly overhead).
- A proper ROC-based similarity threshold (change_detection/build_embeddings.py and detector.py) — right now the three threshold presets in the dashboard (0.85 / 0.70 / 0.55) are reasonable manual choices, not numbers derived from an ROC curve on labeled change/no-change pairs.

None of these require rebuilding anything — they're existing scripts in the project that just need to be run, and their outputs would slot directly into a more complete version of this report.

7. Limitations and Honest Reflection

The biggest structural limitation of this project is that EuroSAT doesn't contain genuine before/after imagery of the same location over time — it's a single-timestamp classification dataset. So the 'change detection' here is really a proxy: it detects when two images produce different class predictions or

dissimilar embeddings, which is a reasonable stand-in, but it's not the same as validating against real temporal satellite pairs (like repeated Sentinel-2 passes of the same coordinates). If I extended this project, that would be the first thing I'd fix — swapping in a real temporal dataset would make the change-detection numbers actually trustworthy rather than illustrative.

On the practical side, getting this running smoothly across two different machines turned out to be its own small project — dependency versions drifting between environments, a virtual environment accidentally pointing at the wrong Python install, a Windows path-length limit that broke installing PyTorch, and a couple of matplotlib API changes that needed small code fixes. None of that is a modeling problem, but it's a realistic reminder that a model isn't 'done' until it reliably runs somewhere other than the machine it was built on.

8. What I Learned

The clearest lesson from this project was watching, very concretely, how much unfreezing the right layers matters — an 80% ceiling from a frozen backbone versus 95%+ after two-phase fine-tuning is a bigger gap than I expected going in. The second lesson was more about engineering than modeling: a solid checkpoint isn't the finish line. Turning it into something usable — a dashboard with clear predictions, an adjustable decision threshold, and a heatmap that actually shows where the model is looking — took at least as much iteration as the training itself, and that's the part that actually makes the project useful to someone who isn't the person who built it.